What Does Formal Structure in Counterexample Feedback Buy?

Anonymous Author(s)
Affiliation
Address
email

Abstract

Counterexample-guided repair asks a synthesizer to fix a program given a description of how it failed. When that synthesizer is a large language model, the description can range from a raw execution trace to a formal certificate carrying a violated predicate, a signed robustness value, a minimized falsifying input, and the program location responsible. We hold the falsifier, the program, the counterexample corpus, the token budget and the evaluation suite fixed, vary only the rendering of the failure across five conditions, and run 300 repair attempts against an interpretable behaviour-tree policy for an unprotected left turn in CARLA.

Feedback format strongly determines what kind of repair is attempted: structural edits account for $59/60$ attempts under a raw trace against $13/60$ under a natural-language summary, a difference of 76.7 points whose 95% interval is $[+61.7, +90.0]$. After excluding refusals we did not detect a reliable improvement in repair success from any feedback format.

Separately, the natural success criterion is trivially satisfiable by refusing to act. 72% of apparent repairs were the vehicle declining to enter the junction, and exactly one of 300 attempts repaired the failure, completed the manoeuvre and broke nothing. That one repair generalises: re-evaluated across all twenty counterexamples, including the nineteen it was never shown, it resolves and completes every one. A safe and task-completing program is therefore expressible in this language and was found once in three hundred attempts, while the degenerate answer was found far more often. The binding constraint is the search rather than the vocabulary, and the hazard we identify is that a specification rewarding only safety makes the degenerate answer the easier one to find.

1 Introduction

Counterexample-guided inductive synthesis is a long-established pattern in program synthesis [Solar-Lezama et al., 2006]. A synthesizer proposes a candidate, a verifier tries to refute it, and each refutation constrains the next proposal. Recent work substitutes a large language model for the synthesizer, and tests or execution feedback for the verifier [Madaan et al., 2023, Shinn et al., 2023, Chen et al., 2024, Orvalho et al., 2025]. In safety-critical control the natural verifier is not a test suite but a falsifier: a search over scenario parameters for an input on which the policy violates a temporal-logic specification.

That substitution raises a design question usually settled by intuition. A falsifier can return a great deal of structure. It can report which predicate was violated, by what signed margin, at which timestep, under which minimal parameter assignment, at which boundary of the failing region, and which part of the program was responsible. None of it is free: delta debugging and boundary search

Submitted to NeurIPS 2026 Workshop on AI for Verifiable Coding. Do not distribute.

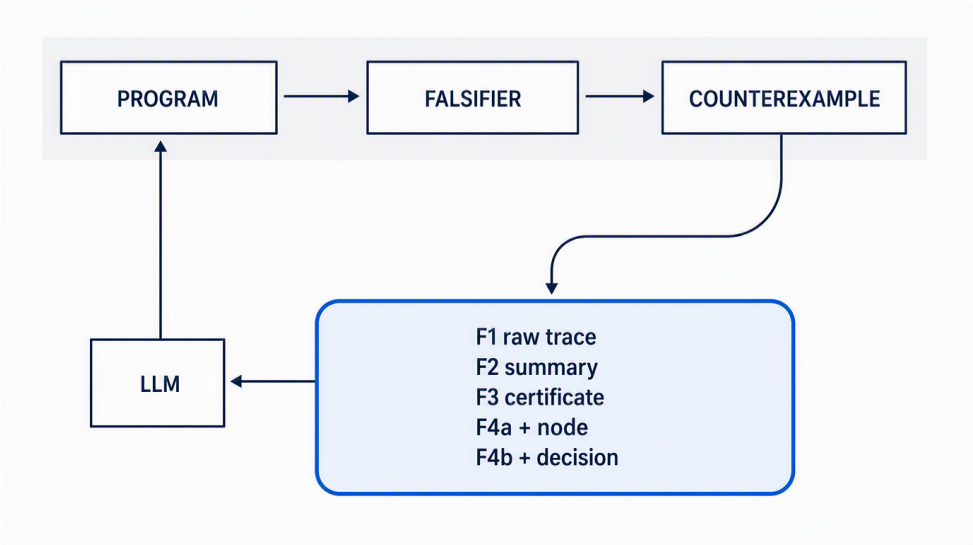


Figure 1: The ablation. Falsifier, frozen program, counterexample corpus, token budget and evaluation suite are identical across all five conditions (grey band). Only the rendering of the failure into text differs (highlighted). The corpus is banked once and read by every condition, so minimization and boundary-search cost are amortized rather than charged to the structured conditions.

cost simulations, and every field costs tokens. The intuition is that more structure repairs better. We set out to test it.

The answer is that structure changes what repair is attempted, that we could not detect it changing whether repair succeeds, and that pursuing the second question turned up a more useful one. Our largest and most robust effect is on the *kind* of edit the model makes. Our most transferable one concerns what repair converges on when the specification asks only for safety.

Contributions.

1. Feedback format determines the kind of repair attempted, with a 76.7-point difference in structural-edit rate between the least and most processed conditions (§4.1).
2. Contract satisfaction on a failing scenario is trivially achievable by refusing to act, and the usual defence of also measuring task completion fails when completion is scored on a different input distribution (§4.2).
3. A constructive account of why, and of what the language does and does not permit. Hand-written repairs isolate three mechanisms that make the degenerate answer easy to reach, while the single clean repair found by the model shows a safe and completing program is expressible and generalises to the whole corpus (§4.3).
4. An absence of any detected improvement in repair success, with intervals that respect the shared counterexample corpus and repeated attempts (§4.4), and a structural precondition, $H(\text{node} \mid \text{contract}) > 0$, for when blame localization can carry information at all (§4.6).

2 Related Work

LLMs that write control programs. Code-as-policies approaches have language models emit executable policies [Liang et al., 2023], with prompt construction following the import-block convention of ProgPrompt [Singh et al., 2023]. These synthesize but do not falsify.

Adversarial scenario generation. Falsification and adversarial scenario search for driving policies are well established [Wang et al., 2021], with declarative scenario specification following Scenic [Fremont et al., 2019]. These falsify, but where they repair at all they do so by retraining weights or augmenting data.

	Contract	Robustness	Threshold
ρ_1	no collision	metres of bounding-box clearance	≥ 0
ρ_2	yield to oncoming	seconds of margin above τ_{yield}	3.0 s
ρ_3	do not enter on red	seconds of green remaining at entry	≥ 0
ρ_4	RSS distance to lead	metres above the safe distance	RSS Lemma 2
ρ_5	kinematic comfort	m/s^2 of headroom	3.0 m/s^2

Table 1: The five contracts, in signal temporal logic over logged trajectories using space robustness [Donzé and Maler, 2010]. Overall robustness is $\rho = \min_i \rho_i$, so the argmin identifies both the weakest contract and the timestep at which it is weakest. ρ_4 uses the same-direction safe longitudinal distance of Shalev-Shwartz et al. [2017]; ρ_2 is that work's rule 3 and the avoidability filter is its rule 5.

Counterexample-guided LLM repair. Closest to this work, Orvalho et al. [2025] run a CEGIS loop with an LLM synthesizer over introductory programming assignments, using a test suite as the oracle and MaxSAT-based fault localization to produce a bug-free program sketch. They find that counterexample guidance substantially improves repair across six models and yields smaller fixes. Their question and ours are complementary rather than competing: they ask whether counterexample feedback helps at all and find that it does, whereas we hold the counterexample fixed and ask whether adding formal structure to its rendering helps further. Our null is therefore not in tension with their positive result. Two differences in setting are worth noting. Their oracle is a test suite over discrete inputs; ours is a robustness falsifier over a continuous scenario space, so a counterexample carries a signed margin and a failing region rather than a single failing input. And their programs are scored only on correctness, whereas ours must satisfy a safety specification *and* complete a task, which is precisely the tension §4.3 turns on.

LLM self-repair. Self-Refine [Madaan et al., 2023], Reflexion [Shinn et al., 2023] and Self-Debugging [Chen et al., 2024] iterate on free-form or test-derived feedback, and two results from that work shaped this design directly. Reflexion's ablation removes the natural-language self-reflection step and finds that performance returns exactly to baseline, 0.60 pass@1 against 0.60 for the base model, where full Reflexion reaches 0.68. Every structured condition here therefore carries a verbalized narrative alongside its structured fields. Self-Debugging reports that adding an execution trace to unit-test feedback *lowers* accuracy on TransCoder, 87.9% against 91.6% for unit-test feedback alone. More structure is evidently not monotonically better, and we recorded in advance that we did not assume it would be.

3 Method

3.1 Task, simulator, and contracts

The task is an unprotected left turn, a permissive circular green rather than a green arrow, at a fixed signalised four-way junction in CARLA [Dosovitskiy et al., 2017] Town05, recorded by exact junction identifier. The simulator runs synchronously at 20 Hz with fixed physics substepping. Sensing is ground-truth state and weather is fixed.

Reproducibility, stated precisely. The ego trajectory reproduces bit-identically under identical scenario parameters, both within a process and across process restarts, and four of the five contracts reproduce exactly. The exception is ρ_4, which varies by up to 0.69 m because the lead vehicle's braking trigger tests a position read back from the simulator rather than integrating its own state; sub-centimetre noise there shifts the tick on which deceleration begins. We report this rather than claiming blanket determinism. ρ_4 was the binding contract in 0 of 300 attempts, so no result below depends on it, but it would corrupt any future result in which it binds.

Conjunction as minimum gives blame localization for free, and masks every contract except the weakest. We therefore report per-contract robustness rather than ρ alone; §4.5 gives a result invisible under the minimum.

Condition	Feedback block	Prompt tokens (med.)
F1	raw trajectory excerpt around the violation	16,381
F2	natural-language narrative only	4,584
F3	contract, ρ, per-contract ρ_i, minimal params, boundary, narrative	4,780
F4a	F3 plus `active_node`	4,871
F4b	F3 plus `authorising_node`	4,867

Table 2: The manipulated variable. All five share one template with a single feedback placeholder; we verify mechanically that the 10,874 bytes of prefix and 102 bytes of suffix are byte-identical across all five conditions and all twenty certificates, splitting by index rather than by string replacement so a feedback block containing scaffolding text could not disguise a difference.

3.2 Program representation and baseline

The policy is a behaviour tree in a small domain-specific language, parsed to an AST we own and compiled to `py_trees`. Owning the AST is what enables tree edit distance, node-level blame, and validation before execution. Composites compile without memory, so every condition is re-evaluated from the root on each tick.

The baseline is LLM-synthesized from one fixed prompt, sampled 20 times, of which 14 were syntactically valid, then *selected* against two gates measured on a shared 100-scenario calibration set: a violation rate in $[0.20, 0.40]$, and task completion above 60%, the latter because a program can reach a low violation rate simply by never entering the junction. The frozen baseline violates on 20.0% of calibration scenarios, enters the junction in 100%, and completes the turn in 88.3%. We report the full distribution of all 14 candidates rather than the selected one alone.

3.3 Falsification, certificates, and the five conditions

The falsifier samples a six-parameter scenario space uniformly, then refines locally around low-robustness points, ranking on per-contract robustness rather than ρ because one parameter moves ρ_1 and ρ_2 in opposite directions and the minimum cancels it. Plausibility constraints are enforced at sample time following Scenic [Fremont et al., 2019]. Each admitted counterexample is minimal (delta-debugged [Zeller and Hildebrandt, 2002]), generalized (each surviving parameter binary-searched to where ρ crosses zero), actionable (a violation-free response existed), and verbalized. We banked $K{=}20$ counterexamples at a mean cost of 26.6 simulations each.

F1 is not a subset of the other conditions in either direction. The raw trace carries the behaviour-tree node columns, so it contains the blame signal F2 and F3 lack, unlabelled and unaggregated but present. At the same time an unwindowed F1 prompt measures 202,812 tokens at the median, $10.1\times$ the per-attempt budget of 20,000, so it must be windowed, and the largest window fitting every certificate (± 10 rows) cannot reach the transition F4b names, which lies 28 to 47 rows back in all twenty cases. The ordering is one of structure, not of information content.

Scope, as planned and as run. The pre-registered plan specified three contracts and four scenario parameters. We ran five and six. The change was made before the baseline was frozen and for a measured reason: on the reference program under three contracts, the violated contract determined the blamed leaf exactly, so blame localization carried no information beyond the contract label and the F3/F4 contrast would have been vacuous by construction. We re-enabled the red-light and lead-distance contracts to widen the range of failure modes, and the parameter space grew to six because two of those parameters are what make the new contracts bind. We report this because it is a deviation from the plan, and because it did not have the intended effect: adding contracts does not by itself break the collapse, since each contract may still map to a single leaf. What addressed it was the alternative blame definition below, which is exploratory and labelled as such throughout. The pre-registered field is `active_node`; `authorising_node` was defined after observing that the first was constant, and no claim in this paper rests on F4b outperforming F3.

Generation settings. All calls use `claude-opus-5` at effort `high`, streaming, with `max_tokens` $= 48{,}000$ and prompt caching on the invariant system block. The model exposes no temperature, `top_p` or `top_k` parameter; `effort` is the only sampling control and is held constant. The served

	F1	F2	F3	F4a	F4b
skeleton edits (of 60)	**59**	13	15	15	28

Table 3: Structural-edit counts by condition. F1 versus F2 is a 76.7-point difference, 95% interval $[+61.7, +90.0]$, from a bootstrap clustered on certificate.

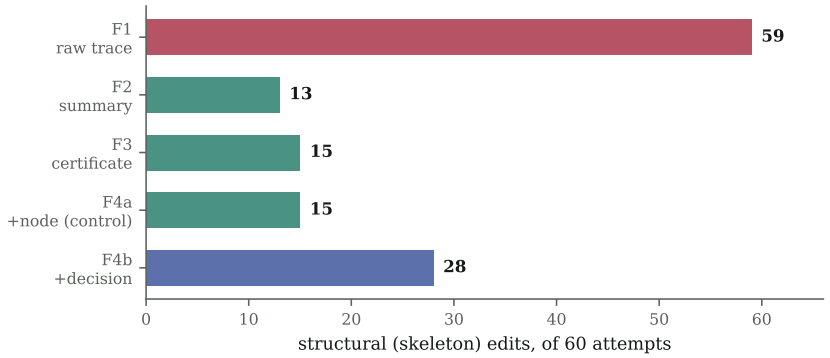


Figure 2: Structural edits by condition, out of 60 attempts each. Raw trajectory data drives the model to restructure the program almost every time; a natural-language summary drives it to adjust a threshold instead.

model identifier is logged per call rather than the requested one. Consequently the replicate index is not a sampling seed: replicates are independent samples, not reproducible ones. We measured 3.00 distinct programs per (condition, certificate) cell out of 3.

Two token limits appear in this paper and they constrain different things. $B = 20{,}000$ is a *prompt* budget, a per-attempt cap on the input we are willing to spend on rendering a counterexample, and it is what forces F1 to be windowed. `max_tokens` $= 48{,}000$ is a *generation* cap on the model's output for a single call, set high enough that no response was truncated; across all 300 attempts none hit it. Neither constrains the other, and no attempt was limited by either.

4 Results

Five conditions, twenty certificates and three replicates give 300 repair attempts. Each patched program is evaluated on 61 scenarios: the counterexample, a 30-scenario regression suite the baseline passes, and a 30-scenario held-out suite drawn from a reserved interior region the falsifier never sampled. Outcomes fall in three buckets never pooled: *truncated* (generation hit the output cap, a harness failure), *malformed* (validator rejection), and *ok*. Across all 300 attempts we observed no malformed and no truncated outputs, so every failure is a valid program that did not fix the problem.

4.1 Feedback format determines the kind of repair

Every patch is a hole fill, changing a threshold, or a skeleton edit, inserting, removing or reordering a node. This is the largest and most robust condition-dependent effect we measure.

Raw trajectory data drives restructuring in 98.3% of attempts; a natural-language summary does so in 21.7%. F4b sits between the others at 46.7%, and the direction is mechanistically sensible, since naming the decision that authorised the failing action points at a branch whereas naming a violated threshold points at a number. We report that as consistent with a mechanism rather than as a resolved difference: its interval against F3 is $[-3.3, +45.0]$ and includes zero.

The robust claim is narrower than "more structure gives more control", and we state it as such. The ordering is not monotone in structure: the least processed condition produces the most structural edits, and the most processed conditions produce among the fewest. What is resolvable at this sample size is that the raw-trace condition is qualitatively different from every processed condition.

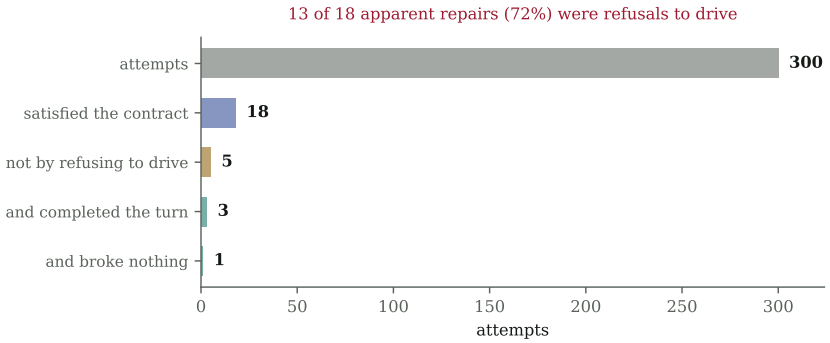


Figure 3: From 300 attempts to one clean repair. The unpatched baseline entered the junction in 20 of 20 counterexample scenarios, so the paralysis is introduced by the patches rather than inherited.

For a system designer that is still directly actionable, because the rendering is a lever on the edit distribution even where it is not a lever on success.

4.2 Contract satisfaction is trivially achievable by refusing to act

The pre-registered success criterion is $\rho \geq 0$ on the failing scenario after the patch. A program that never enters the junction satisfies it for free: ρ_2 and ρ_3 become vacuous, since one cannot fail to yield without entering, and are excluded from the argmin; ρ_1 is satisfied because no actor is ever nearby; and ρ falls through to ρ_5, which a stationary vehicle passes comfortably.

This is the synthesis-side analogue of *vacuous satisfaction* in model checking, where a specification passes for reasons unrelated to the behaviour it was written to constrain, and where detecting such passes is a standard obligation rather than an afterthought [Beer et al., 1997, Kupferman and Vardi, 2003]. Vacuity detection asks whether a formula held trivially; the analogous question for repair is whether a patch satisfied the contracts by making them inapplicable, and on this corpus it usually did.

Thirteen of the eighteen apparent repairs, 72%, are of this kind. Three attempts both satisfied the contract and completed the manoeuvre, and of those three, only one also left the regression suite intact. Across 300 attempts there is exactly one clean repair.

The obvious defence is to measure task completion as well, and it does not work as usually implemented. Our completion metrics were computed over the 60-scenario evaluation suite while success was computed on the single counterexample scenario. In a pilot run, suite completion read 0.93 while the counterexample itself timed out with the vehicle stationary. Nothing was measuring task completion on the scenario the success metric reads from. Any evaluation of repaired safety policies that scores specification satisfaction on one input distribution and task completion on another inherits this defect.

4.3 Why repair converges on refusal, and what the language permits

With no malformed outputs, every failure is a valid program that did not fix the problem, which leaves open whether the language could express a correct repair at all. We tested this constructively, writing candidate repairs by hand in the same vocabulary and running them on all twenty counterexamples.

Three mechanisms make the degenerate answer easy to reach, each measured rather than inferred. First, the baseline's committing branch is guarded only by a stop-line distance test satisfied once the ego is *past* the line, and it outranks the gap-guarded branch, so the gap thresholds are unreachable and raising them changes nothing. Second, the yield predicate is structurally blind to a stationary obstacle: it reports time-to-arrival and, to keep the arithmetic well defined, excludes any vehicle slower than 0.5 m/s, so in one failing scenario a vehicle 9.4 m away at 0.04 m/s is discarded, the predicate returns its 30 s cap, and a guard demanding 15 s of gap passes while that car occupies the junction. Third, the geometric predicate does see it and does block the turn, but the fallback

Repair	Fixed	Completed	Why
frozen baseline	0/20	2/20	
raise gap requirement $5.5 \rightarrow 9.0$	0/20	3/20	branch never reached
gap guard on committing branch, up to 15.0	0/20	2/20	guard passes at every value
geometric guard on committing branch	0/20	2/20	blocks, ego creeps in anyway
geometric guard + true stop	20/20	0/20	never enters the junction
the model's clean repair, from F1	**20/20**	**20/20**	

Table 4: Constructive test of expressiveness. Each row is a valid program in the same DSL run on all twenty counterexamples. The final row is the single attempt of 300 that resolved its counterexample, completed the manoeuvre and caused no regression, re-evaluated here across the whole corpus.

Cond.	Fix rate	Ex-refusal	Fixed & drove	Broke ≥ 1	Mean TED
F1	21.7%	5.0%	1/60	42/60	5.93
F2	1.7%	0.0%	0/60	37/60	5.77
F3	5.0%	3.3%	2/60	44/60	6.95
F4a	0.0%	0.0%	0/60	54/60	7.38
F4b	1.7%	0.0%	0/60	47/60	4.27

Table 5: Repair success over 300 attempts. Every paired difference against F3 has a 95% interval containing zero once refusals are excluded.

decelerates to 1.0 m/s rather than to rest, so the ego creeps across the boundary two seconds later and an unguarded branch commits.

Making the fallback a true stop resolves all twenty counterexamples and completes none of them. That is the degenerate repair, and it is easy to reach.

A safe and completing repair is nonetheless expressible. One attempt in 300, under the raw-trace condition F1, produced a program that resolved its counterexample, completed the manoeuvre and caused no regression failure. Re-evaluated across all twenty counterexamples, including the nineteen it was never shown, **it resolves and completes every one**. Its patch adds a geometric guard to the committing branch and leaves the fallback able to creep, which is sufficient where our hand-written true stop was over-conservative. Its own comment diagnoses the mechanism correctly, noting that a stalled vehicle reports an effectively infinite time-to-conflict-zone so the gap test cannot see it and the path check must reach far enough to.

So the vocabulary is not the binding constraint. The search is. A safe, completing, generalising repair exists in this language and was found once in three hundred attempts, while 72% of apparent repairs converged on the degenerate answer instead. What the specification rewards determines which of the two is easier to find, and rewarding only safety makes the degenerate one easier.

We state the limit plainly. Our hand-written repairs were a family of natural single- and double-edit candidates, not an exhaustive search, so *none of the hand-written repairs tested achieved both across all twenty counterexamples*; we do not claim no such repair exists, and indeed the model found one.

Validation. Because one contract is known to vary by up to 0.69 m over identical parameters (§3), we repeated the clean repair's evaluation on its own counterexample six times. Robustness reproduced exactly, with zero spread in ρ and zero spread in ρ_4, and ρ_4 was never the binding contract. The verdict does not depend on that variability.

4.4 Repair success: no detected improvement

After excluding refusals we did not detect a reliable improvement in repair success from any feedback format. Every paired comparison against F3 has an interval containing zero. We note that an interval containing zero is not evidence that the effect is absent, only that this design did not resolve one. The raw-trace condition clears zero on the pre-registered rate, but ten of its thirteen resolutions are refusals, and excluding those the difference is $+1.7$ with an interval of $[-5.0, +8.3]$.

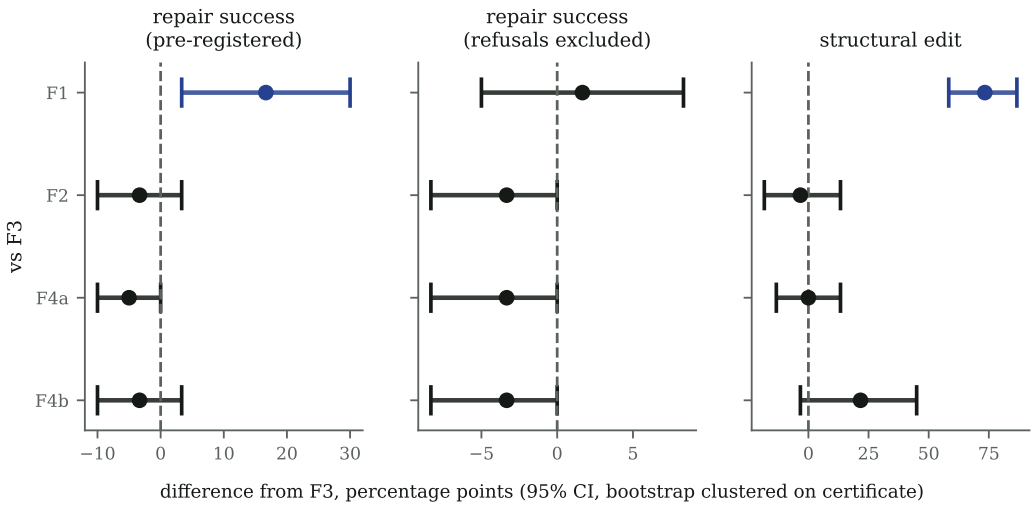


Figure 4: Paired differences against F3 with 95% intervals from a bootstrap clustered on certificate, which respects both the shared counterexample corpus and the three repeated attempts per cell. Only the raw-trace condition separates from zero, and on repair success that separation is an artifact of resolution-by-refusal: excluding refusals it falls inside the interval with everything else.

We make no significance claim, and we do not claim a null. With $K{=}20$ counterexamples, three replicates and between zero and thirteen successes per condition, the design supports neither.

4.5 Repair is net destructive, and the minimum hides part of it

Across all 300 attempts, 224 (74.7%) caused at least one previously-passing regression scenario to fail, against 18 apparent fixes. The modal outcome of a repair attempt is that it did not fix the counterexample and broke one thing that worked. This holds in every condition.

Per-contract reporting reveals a second effect invisible under the minimum: ρ_5 degrades in every condition without exception, from 1.000 to between 0.59 and 0.93. Every feedback format pushes the model toward harder braking and faster commits, and whenever ρ_1 is the argmin this is masked completely. Generalization changes span $+0.021$ to $+0.039$, a 1.8-point spread well inside the intervals of §4.4.

4.6 When blame localization can carry information

Resolving blame to the leaf executing at the argmin is vacuous for any program whose commit branch is guarded by a region test, whenever the violated contracts are evaluated over that same region, because the guard leaves exactly one leaf reachable there. On our baseline $H(\texttt{active_node} \mid \text{contract}) = 0.000$ bits against $H(\text{contract}) = 1.234$ bits. This is a property of program structure and not of contract count: we verified by construction that five contracts each blaming one node still yields $H(\text{node} \mid \text{contract}) = 0$.

We therefore propose a precondition, cheap to check before running any such experiment:

$$H(\text{node} \mid \text{contract}) > 0.$$

Where it fails, the blame field is a deterministic re-encoding of the contract label and carries no information beyond it. We stop short of calling such a condition a placebo: redundant text can still influence a language model's output, which is precisely why we do not treat F4a as a measure of the experiment's resolution. Defining blame instead as the last distinct leaf active before the failing leaf began running, which is the decision that authorised the action rather than the action executing when the consequence landed, raises this to 0.500 bits on the same corpus. We report both, note that the second was defined after observing the first was vacuous, and make no claim that F4b outperforms F3.

5 Limitations

Two limitations deserve more than a clause. First, **every result is conditioned on a single frozen baseline program**. The structural argument of §4.6 generalises, since it depends only on a commit branch guarded by a region test, but no effect size here is claimed to transfer to another program. A second baseline would test that directly and is the most valuable single extension we can identify; it would also tell us whether the refusal attractor of §4.3 is a property of this program's branch ordering or of the task. Second, **the expressiveness audit is not exhaustive**. We wrote a family of natural single- and double-edit repairs and ran them; we did not search the program space. That family failed to find a safe and completing repair, and the model found one, which is direct evidence that the family was incomplete rather than that the language was. Any statement here about what the DSL permits should be read as a statement about what these candidates reached.

The remaining limitations are ordinary. $K{=}20$ with three replicates supports no significance claim, and an interval containing zero is not evidence of absence. We measure one repair step per counterexample rather than compounding effects over rounds. F1 is not a subset of the other conditions in either direction (§3.3), so the ordering is one of structure rather than information content. F1 also differs in *length* as well as in kind, at roughly 16,400 prompt tokens against about 4,700 for every other condition, so the edit-type contrast of §4.1 cannot separate the effect of raw signal from the effect of prompt volume. Seven of twenty certificates report no boundary, because their sole boundary parameter was measured to be anti-correlated with the quantity it names and was suppressed; those degrade to point counterexamples. Corpus construction is amortized across all five conditions, so the cost asymmetry a deployed closed loop would face is absent here. The simulator reproduces bit-identically but the model does not, since it exposes no seed or temperature. Oncoming vehicles travel at constant velocity, making the scenario harsher than naturalistic traffic.

6 Conclusion

Formal structure in counterexample feedback changes what repair is attempted, substantially and robustly, with raw execution traces driving structural rewrites where processed feedback drives threshold adjustments. After excluding refusals we did not detect a reliable improvement in repair success from any feedback format, which at this sample size is a statement about our resolution rather than about the effect.

The more useful finding came from asking why so many apparent repairs stopped driving. Contract satisfaction on a failing scenario is trivially achievable by inaction, and the usual defence of also measuring completion fails when completion is scored on a different input distribution. A safe and completing program does exist in this language, and one attempt in three hundred found it and generalised to the whole corpus. Repair converged on refusal not because the vocabulary forbade the alternative but because the specification made the degenerate answer easier to find.

We suggest this generalises. Where a repair task scores only specification satisfaction, and the program representation admits inaction, counterexample-guided repair should be expected to converge on inaction, and the fix belongs in the task specification rather than in the feedback.

References

Ilan Beer, Shoham Ben-David, Cindy Eisner, and Yoav Rodeh. Efficient detection of vacuity in ACTL formulas. In *Computer Aided Verification (CAV)*, volume 1254 of *Lecture Notes in Computer Science*, pages 279–290. Springer, 1997.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024.

Alexandre Donzé and Oded Maler. Robust satisfaction of temporal logic over real-valued signals. In *Formal Modeling and Analysis of Timed Systems (FORMATS)*, volume 6246 of *Lecture Notes in Computer Science*, pages 92–106. Springer, 2010.

Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio López, and Vladlen Koltun. CARLA: An open urban driving simulator. In *Proceedings of the 1st Annual Conference on Robot Learning (CoRL)*, volume 78 of *Proceedings of Machine Learning Research*, pages 1–16, 2017.

Daniel J. Fremont, Tommaso Dreossi, Shromona Ghosh, Xiangyu Yue, Alberto L. Sangiovanni-Vincentelli, and Sanjit A. Seshia. Scenic: A language for scenario specification and scene generation. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 63–78, 2019. doi: 10.1145/3314221.3314633.

Orna Kupferman and Moshe Y. Vardi. Vacuity detection in temporal model checking. *International Journal on Software Tools for Technology Transfer (STTT)*, 4(2):224–233, 2003. doi: 10.1007/s100090100062.

Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 9493–9500, 2023.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 46534–46594, 2023.

Pedro Orvalho, Mikoláš Janota, and Vasco Manquinho. Counterexample guided program repair using zero-shot learning and MaxSAT-based fault localization. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2025.

Shai Shalev-Shwartz, Shaked Shammah, and Amnon Shashua. On a formal model of safe and scalable self-driving cars. *arXiv preprint arXiv:1708.06374*, 2017. URL `https://arxiv.org/abs/1708.06374`.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. ProgPrompt: Generating situated robot task plans using large language models. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 11523–11530, 2023.

Armando Solar-Lezama, Liviu Tancau, Rastislav Bodík, Sanjit Seshia, and Vijay Saraswat. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 404–415, San Jose, CA, USA, 2006.

Jingkang Wang, Ava Pun, James Tu, Sivabalan Manivasagam, Abbas Sadat, Sergio Casas, Mengye Ren, and Raquel Urtasun. AdvSim: Generating safety-critical scenarios for self-driving vehicles. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9909–9918, 2021.

Andreas Zeller and Ralf Hildebrandt. Simplifying and isolating failure-inducing input. *IEEE Transactions on Software Engineering*, 28(2):183–200, 2002. doi: 10.1109/32.988498.